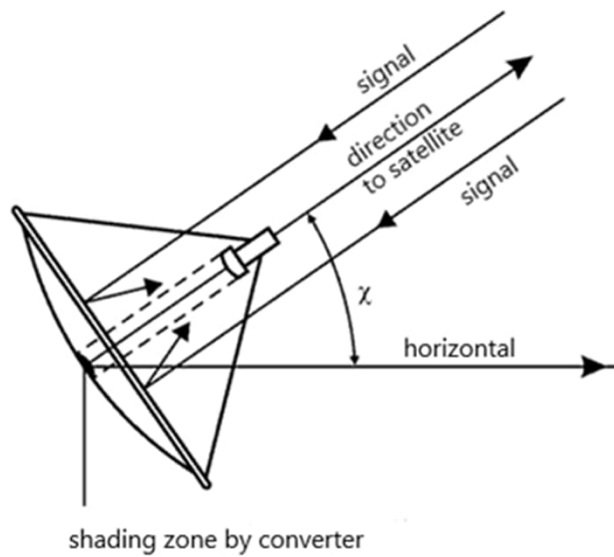


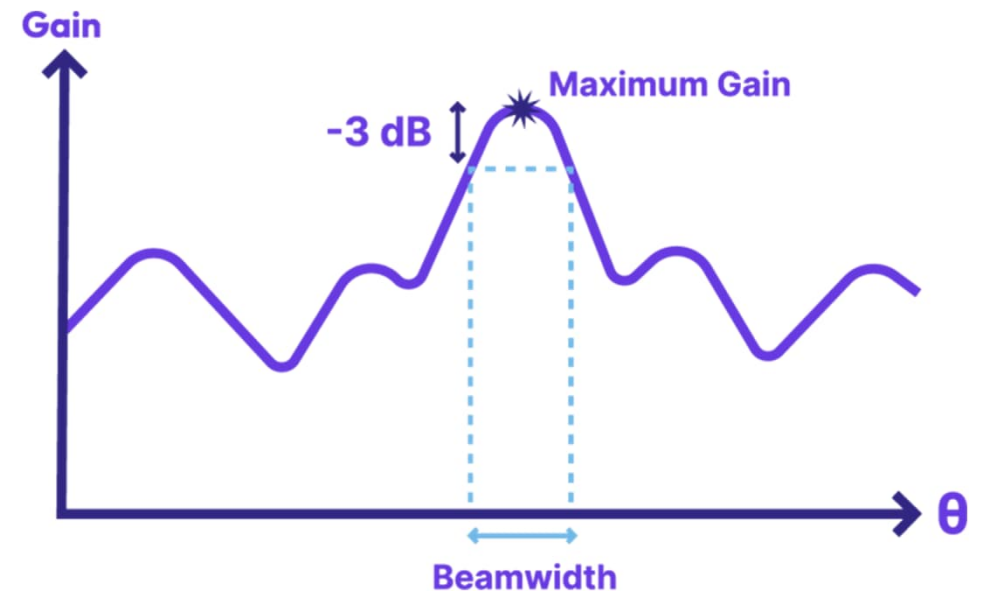
Section 0

Preliminaries: Optimization

Preliminaries (optimization)



e.g. Antenna Gain



Preliminaries (Solution Concepts)

Extremum points



Maximum



Minimum



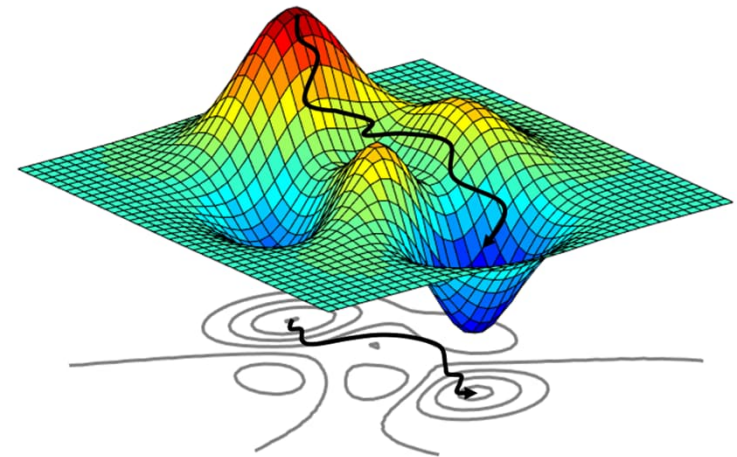
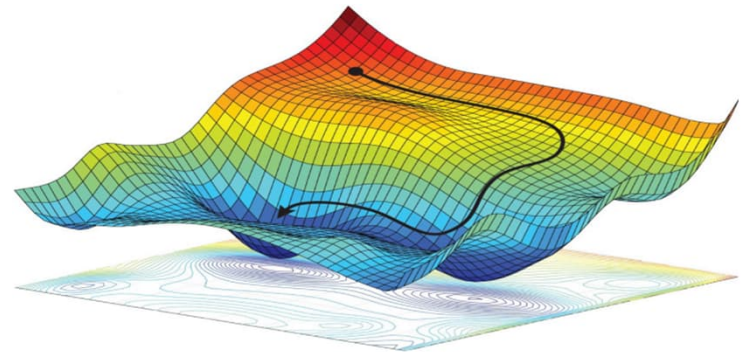
Inflection



Corner



Discontinuity



Preliminaries (Problem Formulation)

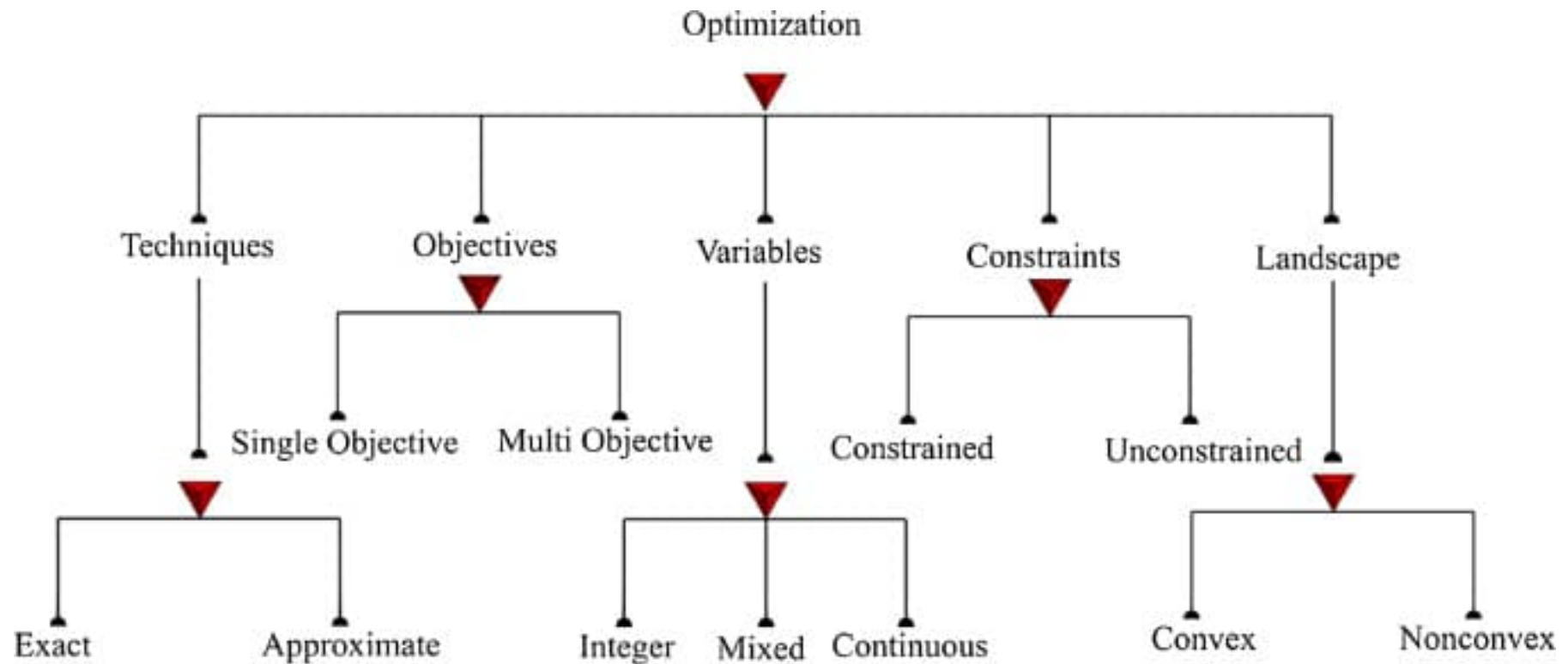
Objective Function تابع هدف

Constraints قيود

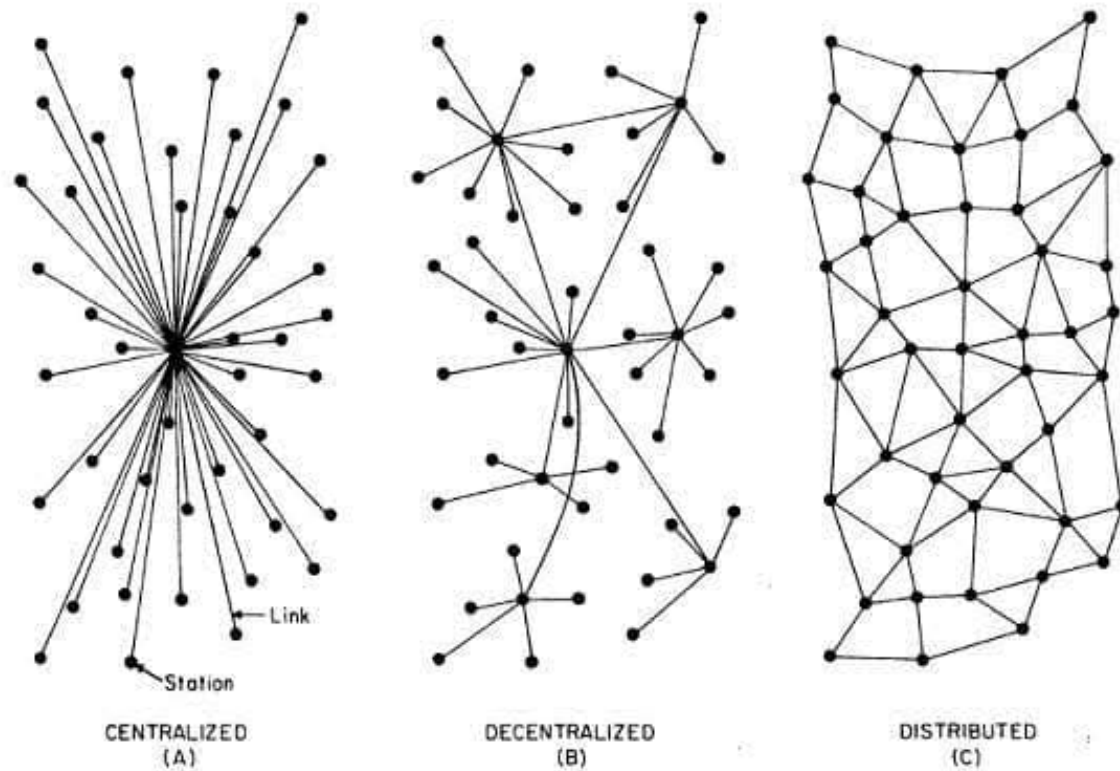
Inequality
Equality
Set

$$\begin{array}{ll}\min_x & f(x) \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \\ & x \in X\end{array}$$

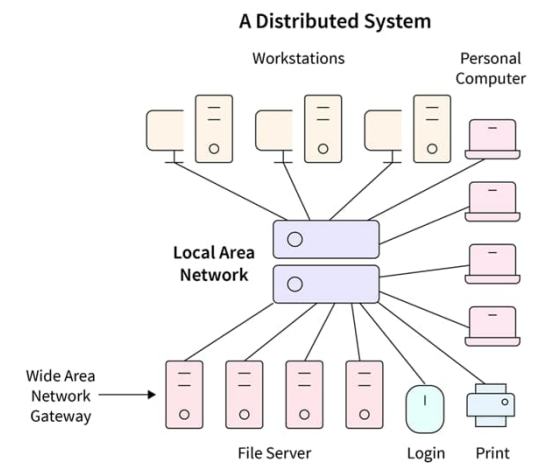
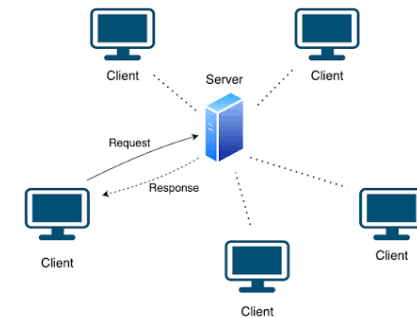
Preliminaries (Problem Formulation)



Preliminaries (Multiple Objectives; Spatial and Computational)

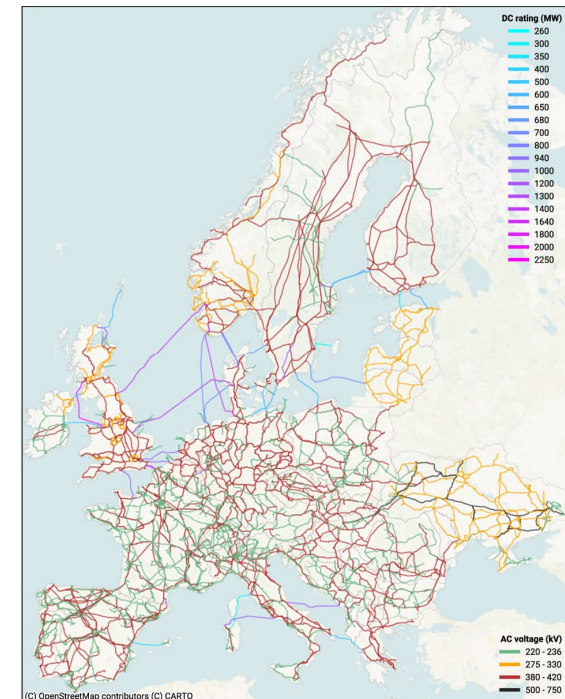
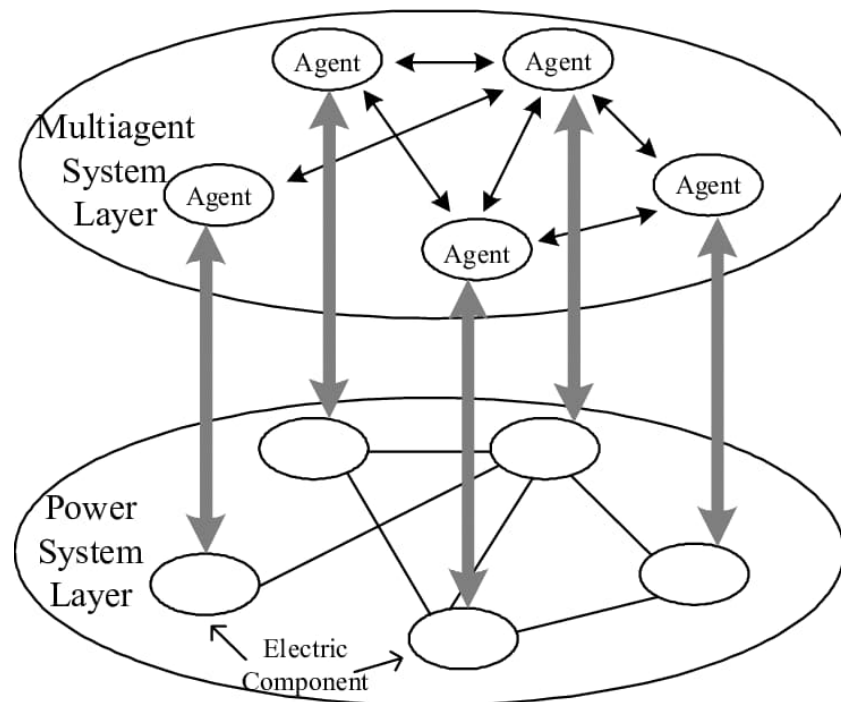


e.g. Network Topology

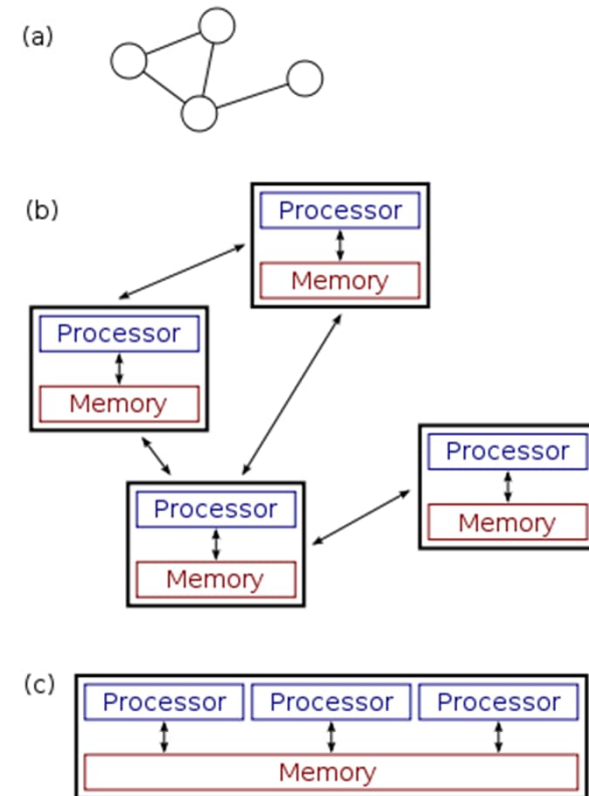
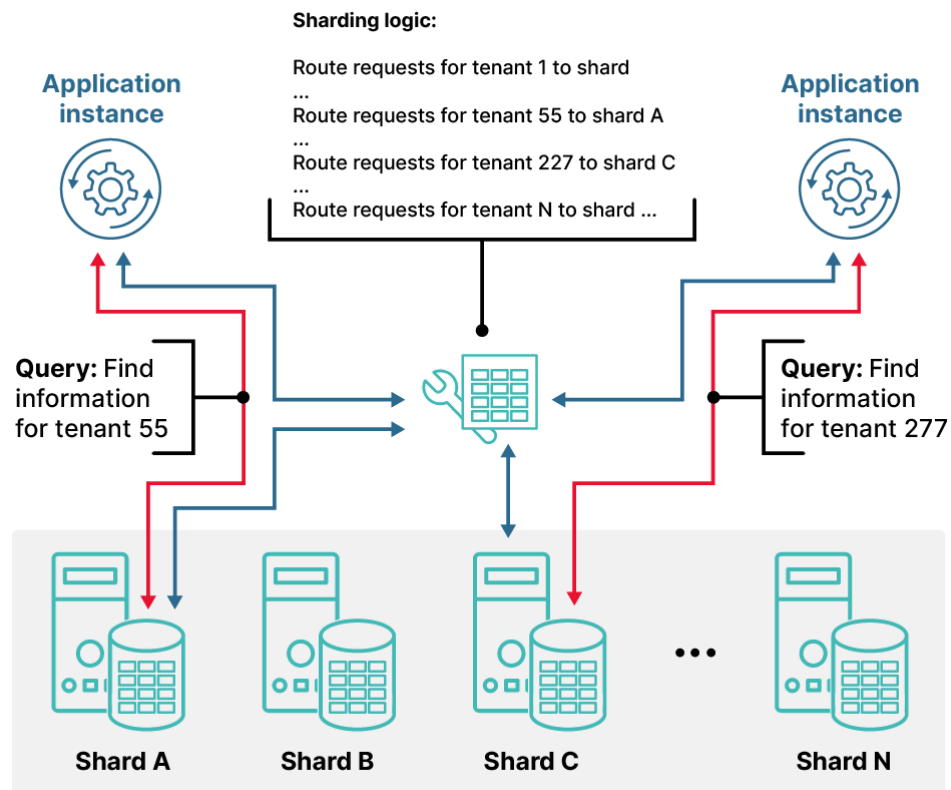


Preliminaries (Multi-agent Systems, Cyber-physical Systems)

e.g. European Power grid (transmission)



Preliminaries (Distributed Systems)



Preliminaries (Some exercises)

Exercise 1 (Constrained Optimization) Formulate some practical problems and computationally solve (python or MATLAB code) in the form of:

- Linear Programming (LP)
- Quadratic Programming (QP)
- Mixed-Integer Linear Programming (MILP)

Preliminaries (Some exercises)

Example 1 (a two-player zero-sum game) Finding the optimal mixed strategy (Nash equilibrium) for a two-player zero-sum game. This is the classic matrix game LP:

$$\max_x \min_j \sum_i x_i A_{ij} \quad \text{s.t.} \quad \sum_i x_i = 1, x_i \geq 0$$

$$\max v \quad \text{s.t.} \quad \sum_i x_i A_{ij} \geq v \quad \forall j, \quad \sum_i x_i = 1, x_i \geq 0$$

$$\min w \quad \text{s.t.} \quad \sum_j A_{ij} y_j \leq w \quad \forall i, \quad \sum_j y_j = 1, y_j \geq 0$$

$$A = \begin{matrix} & \begin{matrix} y1 & y2 & y3 \end{matrix} \\ \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Preliminaries (Some exercises)

Example 1 (a two-player zero-sum game) Finding the optimal mixed strategy (Nash equilibrium) for a two-player zero-sum game. This is the classic matrix game LP:

```
import numpy as np
from scipy.optimize import linprog

# Payoff matrix: rows = Rock, Paper, Scissors; cols = same
A = np.array([[ 0, -1,  1],
              [ 1,  0, -1],
              [-1,  1,  0]])

n = A.shape[0]

# Maximize v <=> minimize -v
# Variables: [x1, x2, x3, v]
c = np.array([0, 0, 0, -1])

# Constraints: for each column j: sum_i x_i A_ij >= v
#              => -sum_i x_i A_ij + v <= 0
A_ub = np.hstack([-A.T, np.ones((n, 1))])
b_ub = np.zeros(n)

# Equality: sum(x) = 1
A_eq = np.array([[1, 1, 1, 0]])
b_eq = np.array([1.0])

bounds = [(0, 1)] * n + [(None, None)]

res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
              bounds=bounds, method='highs')

print("Row strategy x:", res.x[:n])
print("Game value v :", res.x[n])
```

```
A = [ 0 -1  1;
      1  0 -1;
      -1  1  0];
n = size(A,1);

% Variables: [x1 x2 x3 v]
f = [0 0 0 -1];

% -A' * x + v <= 0
A_ub = [-A', ones(n,1)];
b_ub = zeros(n,1);

% sum(x) = 1
Aeq = [1 1 1 0];
beq = 1;

lb = [0 0 0 -inf];
ub = [1 1 1 inf];

[x, fval] = linprog(f, A_ub, b_ub, Aeq, beq, lb, ub);

disp('Row strategy:'), disp(x(1:n))
disp('Game value :'), disp(-fval) % since we minimize
```

```
... Row strategy x: [0.33333333 0.33333333 0.33333333]
Game value v : -0.0
```